문자 수준 CNN과 DistilBERT 기반 캐스케이드 웹 공격 탐지의 속도–정확도 교환 분석

Speed–Accuracy Trade-off Analysis of Cascaded Character-Level CNN and DistilBERT for Web Attack Detection

요 약

인라인 웹 애플리케이션 방화벽(web application firewall, WAF)에서 다중 웹 공격을 탐지할 때, DistilBERT와 같은 사전학습 트랜스포머는 긴 문맥을 포착해 국소 구분자가 유사한 SQL 삽입(SQL injection, SQLi)과 명령 삽입(command injection, CmdI) 간 공격 종류 오탐을 줄이지만 단일 스트림 CPU 추론 지연이 커 실시간 배치에 부담이 되고, 경량 합성곱 신경망(convolutional neural network, CNN)은 짧은 수용 영역이나 전처리 비용의 제약을 받는다. 이를 절충하려는 2단계 캐스케이드와 확신도 기반 넘김(deferral) 기법이 제안되어 왔으나, 전처리를 포함한 종단간 지연과 낮은 실운영 공격 기저율·실사용자 정상 트래픽 분포, 그리고 다중 분할 seed에서도 속도–정확도 이득과 넘김 판단의 유효성이 유지되는지는 검증되지 않았다. 이에 본 논문은 실트래픽 기반 SR-BH 2020 데이터셋에서 사전 등록된 판정 기준에 따라 바이트 이미지 CNN 1차 캐스케이드, 다섯 갈래의 넘김 판단 변형, 그리고 문자 수준 CNN(character-level CNN, char-CNN)과 DistilBERT를 결합한 딥러닝 캐스케이드 및 내부 조기 종료 구조를 단일 스레드 CPU(배치 1) 종단간 지연과 실사용자 정상 트래픽 기준으로 재평가하고 배포 조건별 평가 체계를 제시한다. 실험 결과, 전처리를 포함하고 패딩을 제거한 공정 지연에서 이미지 CNN 1차 캐스케이드는 char-CNN 단독의 **0.27**배로 오히려 느려 속도 이득이 기각되었고, 기본 확신도 게이트는 이상적 오라클 라우터보다 평균 **10.5**배 많은 요청을 넘겼으며 확신도 신호 교체·클래스별 임계값·학습형 라우터·특징 공간 거리 점수 모두 사전 채택 기준을 넘지 못했다. 반면 char-CNN 내부 분기에 보조 헤드를 결합 학습한 조기 종료 구조는 모델 압축 효과로 단독 모델과 동등한 정확도를 **3.5~3.6**배 빠르게 달성하였고, char-CNN → DistilBERT 캐스케이드는 SQLi→CmdI 오탐 감소분의 절반 이상을 흡수하며 재구현 DistilBERT 단독 대비 합산 **10.6~12.3**배 가속을 얻었으나 Macro-F1 손실은 기본 seed 42(**−0.03%p**)에서만 허용 기준을 충족하고 확인용 seed 43(**−0.32%p**)에서는 재현되지 않았다. 결론적으로 웹 공격 탐지 캐스케이드의 병목은 넘김 점수 설계가 아니라 전처리를 포함한 1·2차 지연 격차와 1차 모델의 기본 판별력에 있으며, 캐스케이드의 실효성 주장은 반드시 종단간 지연, 배포 공격 기저율, 실사용자 정상 넘김률, 다중 seed 분산을 함께 보고할 때에만 성립한다.

ABSTRACT

In inline web application firewalls (WAFs), pre-trained transformers such as DistilBERT capture long-range context and reduce attack-type confusion between syntactically similar SQL injection (SQLi) and command injection (CmdI), but their single-stream CPU latency hinders real-time deployment, whereas lightweight convolutional neural networks (CNNs) face either limited receptive fields or preprocessing overhead. Two-stage cascades and confidence-based deferral have been proposed to bridge this gap, yet it remains unverified whether their speed–accuracy gains and deferral mechanisms hold under end-to-end preprocessing latency, low deployment attack base rates with real-user benign traffic, and multiple split seeds. In this paper, we re-evaluate byte-image CNN cascades, five families of deferral-gate variants, and all-deep-learning cascades pairing character-level CNN (char-CNN) with fine-tuned DistilBERT alongside internal early exit on the real-traffic SR-BH 2020 dataset under pre-registered criteria using single-thread CPU (batch 1) end-to-end latency and real-user benign traffic, establishing a deployment-aware evaluation protocol. Experiments show that under fair unpadded end-to-end latency, the byte-image CNN cascade slows to **0.27**× of stand-alone char-CNN (withdrawing its speedup claim), while the standard confidence gate defers **10.5**× more requests than an oracle router and none of the alternative signals, class-wise thresholds, learned routers, or feature-space distance scores pass pre-specified adoption criteria. Conversely, joint-trained internal early exit in char-CNN achieves **3.5–3.6**× speedup at stand-alone accuracy via model compression rather than deferral, and the char-CNN → DistilBERT cascade inherits over half of DistilBERT's SQLi-to-CmdI error reduction while achieving **10.6–12.3**× pooled speedup over stand-alone DistilBERT, though its Macro-F1 loss meets the pre-specified tolerance only on seed 42 (**−0.03 pp**) and fails to replicate on

confirmation seed 43 (**−0.32 pp**). Ultimately, the bottleneck in cascaded web attack detection lies in end-to-end stage latency ratios and first-stage capacity rather than deferral scoring, demonstrating that valid cascade claims require jointly reporting end-to-end latency, deployment base rates, real-user benign deferral, and multi-seed variance.

Keywords: Web attack detection, Cascade inference, Character-level CNN, DistilBERT, Command injection, Inference latency

I. 서론

웹 애플리케이션 방화벽(web application firewall, WAF)은 모든 인바운드 HTTP 요청의 인라인 경로에 놓이므로, 분류 정확도뿐 아니라 요청 한 건당 추론 지연을 함께 충족해야 한다. 동시에 보안 분야의 기계학습 평가에서는 실험실 측정 조건과 실제 배포 환경의 괴리가 성능을 과장한다는 점이 반복해 지적되어 왔다[1].

본 연구는 악성코드 바이트열을 이미지로 변환해 분류하는 기법[2]을 웹 요청에 옮겼을 때 텍스트 신경망과 대등한 성능을 내는지(RQ1)를 검증하는 데서 출발하였다. 교차검증 결과 답은 "아니오"였다(문자 수준 합성곱 신경망(character-level CNN, char-CNN) 대비 Macro-F1 약 9%p 열위). 이 음성 결과를 그대로 보고하고, 가벼운 1차 모델이 대다수 요청을 처리하고 확신도가 임계값 τ 미만인 요청만 무거운 2차 모델로 넘기는 캐스케이드(cascade)가 2차 단독 정확도를 더 낮은 비용으로 얻는지(RQ5: 축 ① 속도–정확도 교환, 축 ② 넘김 판단[3])를 후속 질문으로 검증한다. 이전 단계에서 공개 페이로드 모음과 GPU 배치로 쟀을 때는 5.07배 빨랐으나, 실제 인라인 WAF와 같은 단일 스트림 CPU 환경에서 ① 전처리 포함·무패딩 공정 지연, ② 낮은 배포 공격 기저율(π = 3.3%)과 실사용자 정상 트래픽, ③ 다중 분할 seed 및 지연 유발 공격(slowdown attack)[4]에서도 이득이 성립하는지는 확인되지 않았다.

본 연구는 제안 구성을 원시 바이트열이나 서브워드 토큰을 종단간(end-to-end) 학습하는 딥러닝 모델들로 한정한다(고전 기계학습인 TF-IDF+로지스틱 회귀는 비교 기준선으로만 사용). 실험에서 드러난 핵심 병목은 정상 대 공격의 이진 차단보다, 국소 구분자(;, |, ')를 공유하는 SQL 삽입(SQL injection, SQLi)과 명령 삽입(command injection, CmdI) 간 공격 종류 오탐이었다. 경량 char-CNN은 0.35~0.60ms로 빠르지만 짧은 합성곱 창 때문에 SQLi→CmdI 오탐을 남기고, 사전학습 트랜스포머 DistilBERT[5]는 전역 어텐션으로 이를 줄이는 대신 단일 스레드 CPU에서 약 45~46ms가 걸린다. 실트래픽 기반 SR-BH 2020[6] 위에서 수행한 본 논문의 기여는 세 가지이다.

- 전처리 포함 공정 지연과 배포 조건 재평가. 이미지 1차 캐스케이드가 전처리(0.407ms)와 2차 패딩 제거를 반영한 공정 지연에서 char-CNN 단독의 **0.27**배로 역전됨을 보여 요청 단위 CPU 속도 이득 주장을 철회하고, 표현 이질성이 속도 이득의 필요조건이 아님을 실증한다.
- 넘김 판단의 한계와 배포 기저율 평가 지표 규명. 기본 확신도 게이트는 오라클보다 평균 **10.5**배 많이 넘겼고, 신호 교체·클래스별 임계값·학습형 라우터·거리 점수 4종 모두 사전 기준을 넘지 못했다. 또한 공격이 74.8%인 시험 집합 넘김률만으로 게이트를 고르면 정상 넘김률(r_N)이 늘어 배포 환경에서 더 느려짐을 보이고, 실사용자 r_N(검증 분포 대비 **+24.9%p**)과 π 재가중 조화평균(합산) 배수를 함께 보고하는 기준을 제시한다.
- 딥러닝 캐스케이드**(DL-B·C·A·E14)**의 교환 관계**, CmdI** 오탐 분해 및 공격면 분석. 선행 연구(WAMM[7])의 DistilBERT를 512 토큰 및 원 설정(128 토큰, E12)으로 재구현해 비교한다. char-CNN 내부 조기 종료(DL-C)는 결합 학습된 보조 헤드의 모델 압축 효과로 **3.5~3.6**배 가속을 달성했고, char-CNN → DistilBERT(DL-A)는 SQLi→CmdI 오탐 감소분의 56~74%를 흡수하며 합산 **10.6~12.3**배 가속을 얻되 seed 42(**−0.03%p**)와 seed 43(**−0.32%p**)에서 허용폭(0.30%p) 충족이 갈리는 한계를 보고한다(결합 학습 k3 출구 결합 E14는 DL-A 대비 2.3~2.4배 추가 가속). 아울러 URL 인코딩 1회로 공격 지연이 6.4배 증폭되는 공격면(E15)을 규명한다.

결과에 맞춘 사후 기준 변경(HARKing)을 막기 위해 모든 판정 규칙은 측정 전 버전 관리에 고정하였으며(커밋 2a456d7~d9d3678), 기준 외 분석은 사후 분석 또는 병기로 구분한다.

II. 관련 연구

Nataraj 등[2]은 악성코드 바이트열을 회색조 이미지로 변환해 분류하였고, 이후 취약점 탐지[8]와 RGB 어셈블리 시각화[9]로 확장되었다. 반면 웹 공격 탐지의 주류는 HTTP 문자열을 1차원 시퀀스로 다루며[10], Zhang 등[11]의 char-CNN은 토크나이저 없이 원시 바이트에서 지역 n-gram을 추출하는 경량 기준선으로 쓰인다. Sureda Riera 등[6]의 SR-BH 2020은 실운영 ModSecurity 로그 기반 다중 공격 데이터셋으로, 이 위에서 Osama 등[7]은 WAMM 프레임워크와 DistilBERT[5] 성능을 보고하였고 이중 채널 BERT[12]와 라벨 정제 기반 이상 탐지[13]가 후속 연구로 제안되었다.

캐스케이드는 최대 소프트맥스 확률(maximum softmax probability, MSP)[14][15] 임계값으로 경량 1차와 대형 2차를 잇는 구조로, Tasdemir 등[16]이 SQLi 탐지에서 BERT 대비 약 20배 가속을 보고하였다. 넘김 판단은 선택적 분류[17] 및 넘김 학습[18]과 맞닿아 있는데, Enomoto와 Eda[19]는 온도 보정[20]의 한계를 지적했고 Jitkrittum 등[3]은 2차가 특정 하위 분포에만 강할 때 1차 확신도 게이트가 구조적으로 실패함을 증명하였다. 단일 모델 내부에 보조 출구를 두는 조기 종료[21]는 전처리 중복을 없앤 대안이며, 마할라노비스 등 특징 공간 거리 점수[22]도 후보이나 Jaeger 등[23]은 오분류 탐지에서 단순 MSP가 거리 점수보다 나음을 보였다. 마지막으로 적응형 추론은 지연 유발 공격(DeepSloth)[4]에 노출되며, 단일 스트림 지연 분리 측정[24]과 낮은 공격 기저율(base rate)[25] 반영이 필수적이다.

III. 방법

3.1 입력 표현과 비교 모델

HTTP 요청의 URI와 본문을 이어 붙인 원시 바이트열 $b = (b_0, ..., b_{L-1})$에 대해, 이미지 모델(**RGB CNN**)은 앞 2,304바이트를 48×48 격자에 행 우선(`x[r, c] = b[48r + c]`)으로 배치하고 원시 바이트 값(R), ASCII 문자 범주(G), 5바이트 지역 엔트로피(B)의 3채널로 구성한다(Fig. 1). 다만 SR-BH 2020 요청 길이 중앙값은 96바이트(p95 233, p99 349)여서 격자의 90% 이상이 0 패딩으로 남는다. 대조용 **1-D CNN**(E2)은 같은 3채널 수열을 접지 않고 3×2,304 1차원으로 읽는다.

문자 수준 모델(**char-CNN**[11])은 바이트열을 패딩 없이 실제 길이 L 그대로 임베딩(64차원)한 뒤 커널 3·5·7 1차원 합성곱(각 128채널)과 최대 풀링을 병렬 수행한다. 딥러닝 캐스케이드 탐색을 위해 ① 임베딩 32·커널 3(64채널)으로 줄인 초경량 **char-CNN-tiny**(DL-B 1차), ② char-CNN의 커널 3 분기 뒤에 보조 분류기를 달아 주 손실과 결합 학습($L = L_{main} + 0.3\,L_{aux}$)한 조기 종료 모델 **char-CNN-EE**(DL-C, 그리고 보조 출구만 떼어 DistilBERT와 잇는 **E14**의 1차), ③ 보조 출구와 같은 크기를 단독 학습한 대조군 **char-CNN-k3**를 구성하였다. 2차 트랜스포머인 **DistilBERT**[5]는 WAMM[7]을 동일 분할·CPU에서 비교하기 위해 미세조정하였으며, 원 논문의 128 토큰 제한이 전체의 6.9%(15,553행; CodeInjection 24.7%, CmdI 26.5%)를 잘라내므로 주 실험(DL-A)은 잘림이 없는 512 토큰을 쓰고 128 토큰 원 설정은 E12에서 별도 대조한다(Table 1).

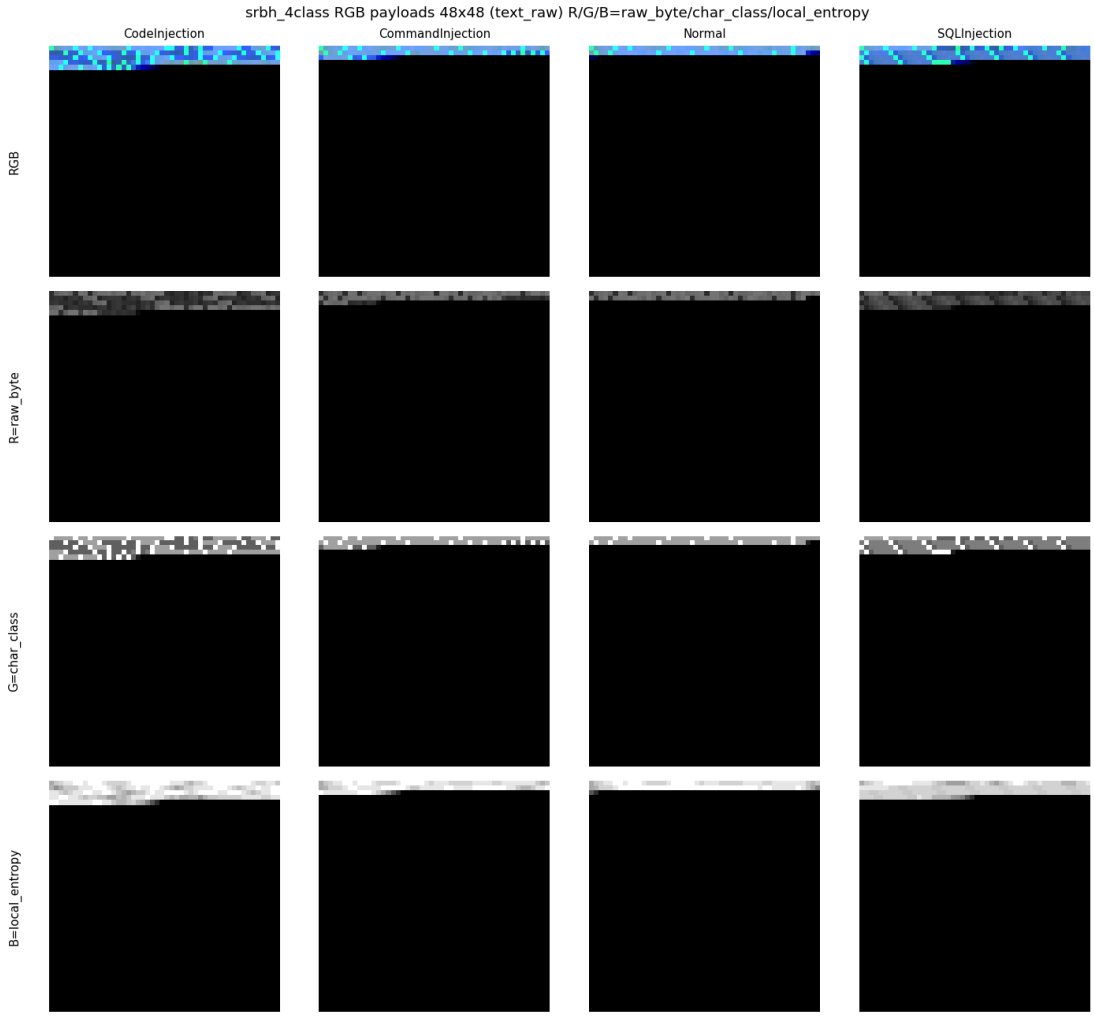


Fig. 1. Examples of 48×48 RGB byte images from SR-BH 2020 web requests (R: raw byte, G: character class, B: local entropy).

Table 1. Models compared in 5-fold cross-validation

Model	Role & setup	Input	Params
RGB CNN	RQ1; image 1st stage	48×48×3 grid	93,988
1-D CNN	E2 1st-stage redesign	3×2,304 1-D	46,660
char-CNN	Ref.; 2nd stage; DL-A 1st	raw bytes (L)	141,252
char-CNN-tiny	DL-B 1st stage	raw bytes (L)	14,692
char-CNN-k3	Single-branch control	raw bytes (L)	41,668
char-CNN-EE	DL-C early exit; E14 1st	raw bytes (L)	141,768
DistilBERT	DL-A/E14 2nd; WAMM[7] ref.	≤512 (or 128) tok.	66.96M
TF-IDF + LR	Classical baseline only	char 2–4-grams	80,004

Notes: All models use class-weighted cross-entropy (early stopping patience 5 on internal validation Macro-F1; DistilBERT fine-tuned for 3 epochs, lr 2e-5, batch 16).

3.2 넘김 게이트 변형과 배포 비용 배수 정의

1차 모델 h_1의 최대 소프트맥스 확률 $s(x) = \max_k p_1(k|x)$가 임계값 τ 이상이면 $h_1(x)$를 출력하고, 미달하면 2차 모델 $h_2(x)$로 넘긴다. 기본 단일 임계값 게이트(**B0**)는 폴드별 내부 검증 집합에서 검증 Macro-F1이 2차 단독의 0.5%p 이내인 지점 중 배포 넘김률 E_π가 최소인 τ를 골라 시험 폴드에 1회 적용한다. B0의 과잉 넘김 원인을 가리기 위해 네 갈래 변형을 대조하였다: ① 신호 교체**(G1)**: 온도 보정 MSP(msp_T)[20], 상위 두 확률 차이(margin), 음의 엔트로피(entropy); ② 예측 클래스별 임계값**(G2)**: 1차 예측 클래스 $\hat{c}_1(x)$별로 τ_c를 달리 두어 내부 검증에서 단위 넘김당 F1 상승이 큰 클래스부터 예산을 배정; ③ 학습형 라우터**(G3)**: $I[h_1(x) \neq y \wedge h_2(x) = y]$를 직접 예측하는 로지스틱 회귀(통계 특징만 쓰는 G3-lite 및 1차 128차원 특징을 결합한 G3-full); ④ 특징 공간 거리 점수**(G6)**: 1차 128차원 특징 공간에서 추정한 마할라노비스(maha)[22], 상대 마할라노비스(rmaha), k=50 최근접 이웃(knn), Trust Score(trust). 게이트 자체의 선별 효율은 오라클 라우터(1차 오답만 넘김, 넘김률 e_o) 대비 2차 단독 F1 도달 최소 넘김률(e_g)의 비인 과잉 넘김 배율 $\boldsymbol{\rho = e_g / e_o}$로 잰다.

1·2차 지연을 t_1, t_2, 기준 모델 지연을 t_{ref}라 할 때, 배포 공격 기저율 $\boldsymbol{\pi = 3.3\%}$[16]에서의 기대 넘김률 E_π와 요청당 기대 지연 t, 그리고 5개 폴드 평균 비용 기준의 합산 속도 배수 $\bar{\mathbf{S}}$**(**폴드 배수 $\mathbf{S_f}$의 조화평균**)**는 다음과 같다.

$$E_\pi = \pi \cdot r_A + (1-\pi) \cdot r_N \tag{1}$$

$$t = t_1 + E_\pi \cdot t_2,\ \bar{S} = t_{ref} / \left((1/5) \Sigma_f t_f \right) \tag{2}$$

산술평균 배수는 넘김이 적은 특정 폴드가 전체 평균을 부풀리므로, 본 논문은 조화평균인 합산 배수 $\bar{S}$를 헤드라인 지표로 쓰고 산술평균은 괄호에 병기한다.

IV. 실험 환경

데이터셋 및 실사용자 정상 층 분리**.** SR-BH 2020[6] 원본 907,815행을 Normal, SQLi, CodeInjection(서버 측 PHP 실행 탐침 위주), CmdI의 4클래스로 매핑하고, 시그니처 감사로 적발된 정상 라벨 오입 행(30,280건)·충돌 행·완전 중복 행을 제거해 총 265,939행(학습 186,157 · 검증 39,891 · 봉인 시험 39,891)을 구성하였다. 입력은 URI+본문(F2)을 사용한다(쿠키 추가 F3은 마스킹 대조 F3M과 보정 p = 0.17로 유의차 없음). 최소 클래스인 CmdI는 전체의 2.1%(학습+검증 풀 3,886건)이다. 특히 Normal 요청의 97.8%는 취약점 스캐너(Nikto 등)의 자동 탐색 요청이었고, 일반 브라우저 User-Agent의 실사용자 정상 요청은 학습+검증 풀**(226,048**행**)** 내 **550**건**(**폴드당 약 **110**건**)**뿐이었다. char-CNN 기준 실사용자 정상 오탐률(false positive rate, FPR)은 2.73%로 스캐너 정상(0.36%)의 7.6배였으므로, 모든 평가에서 실사용자 정상 층과 스캐너 층을 분리하고 주 판정은 실사용자 r_N 기준으로 수행한다.

교차검증**,** 공정 지연 측정 및 사전 판정 기준**.** 봉인 시험 집합을 뺀 **226,048**행 풀에서 동일한 층화 5-fold 교차검증을 수행하며(주 지표는 **Macro-F1**), 유의성 검정에는 학습 집합 중복을 보정하는 Nadeau–Bengio 보정 t-검정[26]과 Holm 보정을 적용한다. 지연은 단일 데스크톱(Ryzen 7 9800X3D, RTX 4080 SUPER, PyTorch 2.6.0)의 **CPU** 단일 스레드·배치 **1** 조건에서 전처리부터 예측까지의 종단간 시간(2,000건×30회 반복 p50)을 잰다[24]. 실행 간 클럭 표류(4~7%)를 통제하기 위해 비교 모델들은 반드시 같은 실행 안에서 번갈아 재고 같은 실행 값끼리만 배수를 낸다. 기준 모델 대비 우위("위") 사전 판정 기준은 ① 실사용자 합산 배수 $\bar{\mathbf{S}} \geq \mathbf{1.2}$**배,** ② **5**개 폴드 모두 $\mathbf{S_f > 1.0}$**배,** ③ **Macro-F1** 손실 ≤ **0.30%p** 세 조건의 동시 충족이다.

V. 실험 결과

5.1 이미지 단독 및 1차 캐스케이드의 속도 이득 철회(RQ1 · 축 ①)

RQ1(음성 결과**)** 및 **1-D** 대조**(E2).** 5-fold 교차검증(seed 42)에서 RGB CNN의 단독 Macro-F1은 **0.8949**로 char-CNN(**0.9884**)보다 9.35%p 낮았다($p < 0.001$). 동일한 3채널 수열을 2차원으로 접지 않고 1차원으로 읽은 **1-D CNN**(E2)은 F1이 **0.9341(+3.92%p, Holm p = 0.009)**로 상승하여, 성능 저하의 주원인이 48바이트마다 줄바꿈하며 인접 문맥을 끊는 2차원 재배치 자체임을 확인하였다.

공정 지연**(E1)**에서의 역전과 속도 이득 철회**.** 과거 2,304바이트 패딩·순전파만 잰 채움 기준에서는 이미지 1차 캐스케이드가 고정 분할 시험 비율에서 1.22배(클래스 가중치 학습 시 1.63~1.67배, π = 3.3%

에서 1.42~1.45배; 교차검증 검증 분포 1.34배, 실사용자 1.01배; 이전 데이터셋 GPU 배치 트랙은 5.07배)로 보였다. 그러나 전처리를 포함하고 2차 패딩을 없앤 공정 지연(E1)으로 재면, **1**차 **RGB CNN** 전체 지연은 **1.311ms(**이미지 변환만 **0.407ms** + 순전파 **0.879ms)**인 반면 실제 길이(중앙값 96바이트)를 읽는 **2**차 **char-CNN** 전체 지연은 **0.393ms**에 불과했다(패딩 시 2.990ms로 7.6배 부풀려짐). 이미지 변환 시간(0.407ms) 자체가 이미 char-CNN 전체(0.393ms)보다 길기 때문에 같은 변환을 쓰는 1차는 2차보다 빨라질 수 없으며, 실제 캐스케이드 배수는 char-CNN 단독의 **0.27**배(실사용자 0.25배; 1-D CNN 1차로 넘김을 43.7%→29.8%로 줄여도 **0.28**배)로 오히려 느렸다. 따라서 요청 단위 CPU 처리에서의 속도 이득 주장을 철회한다. 한편 고전 선형 1차(TF-IDF+LR, 0.295ms, E3)는 같은 조건에서 이미지 1차보다 4.7배 빨라 검증 분포 1.24배(실사용자 0.86배)를 냈고, 오류 독립성(Yule의 Q[27]: RGB 0.954 대 선형 0.996)과 무관하게 운영점 F1(0.983~0.984)은 동일하여 표현 이질성이 속도 이득의 필요조건이 아님을 확인하였다.

5.2 넘김 판단(게이트)의 한계와 기저율 함정(축 ②)

RGB CNN 1차 → char-CNN 2차 위에서 기본 확신도 게이트(B0)는 오라클 라우터보다 평균 **10.5**배(폴드 6.1~17.7배) 많은 요청을 넘겼으며, 네 갈래 게이트 변형 모두 사전 채택 기준(과잉 넘김 20% 이상 감소 및 $p < 0.05$)을 넘지 못했다(Table 2).

Table 2. Deferral-gate variants in 5-fold CV (RGB CNN → char-CNN, seed 42)

Gate variant	Metric & change vs. B0	Corr. p	Pre-specified verdict
G1: msp_T / margin / entropy	Excess ρ: −4.3% to −4.7%	0.96	Rejected (need ≥ 20% cut)
G2: Class-wise τ (seed 42)	Excess ρ: −18.7%	0.42	Rejected (power 7–10%)
G2: Confirm (seeds 43–45)	Curve 3/3 above; S +15.7%	0.17–0.41	Curve above; op. unconfirmed
G3: Learned router (128-D)	Excess ρ: −5.9%	0.69	Rejected (need ≥ 20% cut)
G6: Distances (4 variants)	Deferral E_π: +46% to +128%	≤ 0.046	Rejected (significantly worse)

Notes: B0 mean excess deferral ratio ρ = 10.53. Failure-detection AUROC was 0.938 for MSP vs. 0.20–0.87 for distance scores.

신호 교체(G1)는 B0와 순위 상관계수가 0.997 이상이어서 4~5% 감소에 그쳤고, 학습형 라우터(G3)도 5.9% 감소로 기각되었다. 특히 특징 공간 거리 점수 4종(G6)은 오답이 특징 공간 바깥(OOD)이 아니라 결정 경계 근처에 몰려 있어[23] 오히려 넘김을 **46~128%** 증가시켰다. 클래스별 임계값(G2)은 확인용 seed 43~45 세 번 모두 경계 곡선이 B0 위에 놓였으나 유의성($p = 0.17$~0.41)과 운영점 이득(+15.7%, 기준 20%)이 미확인되었다. 즉 이 데이터에서 넘김 점수는 병목이 아니다.

방법론 교훈(기저율 함정과 실사용자 r_N). 공격 비율이 74.8%인 고정 시험 집합에서 전체 넘김률을 줄여 통과했던 G2(−8.8%p)와 G3-full(−6.7%p)은 공격 넘김(r_A)을 줄인 대신 정상 넘김(r_N)을 67.2%에서 71.0%·78.1%로 높여, $\pi = 3.3\%$ 배포 배수에서는 B0(1.02배)보다 오히려 더 느린 게이트(0.99배·0.91배)가 되었다. 또한 교차검증에서 실사용자 정상 층의 **r_N(69.3%)**은 스캐너 위주 검증 분포 **r_N(44.4%)**보다 **+24.9%p** 높았다($p = 1.7\times10^{-5}$, H-G4 채택). 따라서 게이트 평가는 반드시 실사용자 r_N과 π 재가중 합산 배수를 함께 보고해야 한다.

5.3 딥러닝 캐스케이드(DL-B · DL-C · DL-A · E14)와 G2 재확인(E13)

층별 지연 분해 관문(E7)에서 char-CNN 지연의 **91.0%**가 합성곱 층, 8.5%가 호출 고정비로 확인되어 앞 분기 조기 종료의 물리적 타당성이 입증되었다. 동일 5-fold와 CPU에서 측정한 딥러닝 구성들의 결과는 Table 3 및 Fig. 2와 같다.

Table 3. Speed–accuracy trade-off of deep-learning cascades vs. stand-alone references (5-fold CV, CPU batch 1, 1 thread, unpadded end-to-end latency, real-user benign at π = 3.3%)

Configuration	Reference R (F1)	Seed	Gate	Macro-F1 (ΔR)	r_N / r_A	Pooled $\bar{S}$ (arith.*)	Fold S_f range	Verdict vs. R
DL-B: charcnn_tiny → char-CNN	char-CNN (0.9884)	42	B0	0.9839 (−0.45 pp)	30.2% / 4.0%	2.13× (2.39×*)	1.52–4.10×	No (F1 loss)
DL-C: charcnn_ee (internal exit)	char-CNN (0.9884)	42	B0	0.9874 (−0.10 pp)	1.1% / 0.4%	**3.52**× (3.53×*)	3.24–3.62×	**Yes**
DL-C: charcnn_ee (confirm.)	char-CNN (0.9862)	43	B0	0.9895 (+0.33 pp)	0.0% / 0.005%	**3.62**×	3.62–3.62×	**Yes (2/2)**
Control: charcnn_k3 (solo)	char-CNN (0.9884)	42	—	0.9748 (−1.36 pp)	0.0% / 0.0%	3.62×	3.62–3.62×	No (F1 loss)
char-CNN solo vs. DistilBERT	DistilBERT (0.9914)	42	—	0.9884 (−0.303 pp)	0.0% / 0.0%	80.2×	80.2–80.2×	No (0.003 pp short)
DL-A: char-CNN → DistilBERT	DistilBERT (0.9914)	42	B0	0.9911 (**−0.03 pp**)	7.1% / 0.7%	**12.3**× (51.6×*)	2.95–80.2×	**Yes (seed 42 only)**
DL-A: confirm. (E11)	DistilBERT (0.9947)	43	B0	0.9914 (**−0.32 pp**)	8.9% / 0.6%	**10.6**× (38.3×*)	3.93–80.2×	No (p = 0.038)
DL-A + G2 (secondary)	DistilBERT (0.9914)	42	G2	0.9920 (+0.06 pp)	0.7% / 0.7%	51.1× (57.5×*)	32.5–80.2×	Secondary (E13 failed)
DL-A + G2 (secondary)	DistilBERT (0.9947)	43	G2	0.9920 (−0.27 pp)	1.1% / 0.6%	42.5× (50.6×*)	13.7–80.2×	Secondary (E13 failed)
E14: k3 exit → DistilBERT	DL-A B0 (12.2×)	42	B0	0.9883 (−0.28 pp)	—	**29.3**× (×2.40)	—	**Yes (vs. DL-A)**
E14: confirm.	DL-A B0 (10.5×)	43	B0	0.9919 (+0.05 pp)	—	**23.8**× (×2.27)	—	**Yes (2/2 vs. DL-A)**

Notes: Same-run p50 latencies (never mixed across runs): E8 run has charcnn_tiny t_1 = 0.0606 ms, char-CNN t_2 = 0.3451 ms, k3 front t_{front} = 0.0954 ms; E10 run has char-CNN t_2 = 0.563 ms, DistilBERT(512) t_R = 45.15 ms (80.2×); E12 run (for E12–E15) has t_2 = 0.604 ms, t_{front} = 0.159 ms, $t_R(512)$ = 46.10 ms, $t_R(128)$ = 46.08 ms.

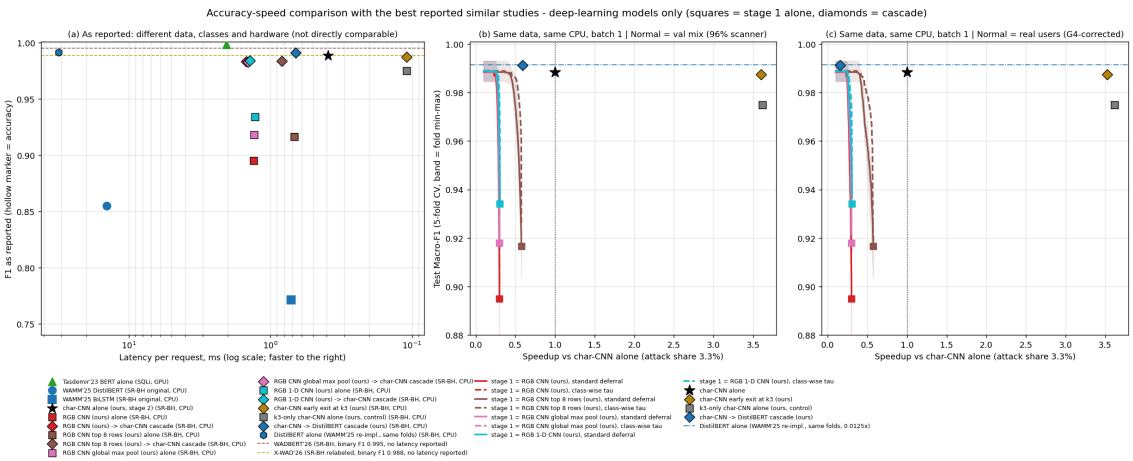


Fig. 2. Accuracy–latency trade-off of deep-learning configurations on SR-BH 2020 (5-fold CV, CPU batch 1, 1 thread).

- **DL-B**: 초경량 1차는 넘김이 30.2%로 늘어 합산 **2.13**배에 그쳤고 F1이 **−0.45%p** 떨어져 기준을 못 넘었다.

- **DL-C(2차 모델 압축)**: char-CNN 내부의 k3 분기에 보조 헤드를 결합 학습하면 두 seed 모두 char-CNN 단독과 같은 정확도를 **3.5~3.6**배 빠르게 냈다. 단, seed 42 다섯 폴드 중 셋과 seed 43 전 폴드가 넘김 0%여서 이득의 원천은 넘김이 아니라 보조 헤드 자체**(2**차 모델 압축**)**다. 같은 크기를 혼자 학습한 charcnn_k3(seed 42, 0.9748)는 보조 헤드보다 1.01%p 낮았는데(보정 p = 0.34; 최적 에폭은 seed 43 보조 헤드가 12~19, seed 42는 보조 헤드 8·10·2·19·3 대 k3 단독 3·20·9·6·2), 인내를 늘린 대조(E16)는 미실시하여 결합 학습의 몫인지 조기 종료 안정성인지는 가리지 못했다.
- **DL-A(**주 결과**)**와 **seed 43** 미재현: char-CNN → DistilBERT는 실사용자 합산 배수로 재구현 DistilBERT 단독보다 **10.6~12.3**배 빨랐고(두 seed 모두 전 폴드 > 1배), F1 손실은 **seed 42 −0.03%p, seed 43 −0.32%p**였다. 사전 기준(손실 0.30%p 이내)은 **seed 42**에서만 충족했고 확인용 seed 43에서는 0.02%p 차로 재현되지 않았다. seed 43에서는 캐스케이드 F1(0.9911→0.9914)이 유지된 반면 기준인 DistilBERT 단독이 3 에폭 미세조정 편차로 0.9914→0.9947로 올라갔는데, 트래픽의 약 93%를 1차 판정으로 끝내는 구조상 2차가 강해져도 상한이 묶인다(char-CNN 단독 대비로는 지연 6.5~7.6배에 F1 +0.28·+0.53%p이나 보정 p = 0.34·0.23으로 유의성 미확인).
- 느린 폴드 진단**(A2)** 및 클래스별 **τ** 새 **seed** 확인**(E13)**: seed 42 폴드 4의 배수가 2.95배로 낮았던 것은 검증 F1을 맞추려 τ = 0.985(예산 3.0%)까지 올리면서, 1차가 Normal로 맞게 예측했으나 덜 확신한 실사용자 정상 요청(36건 중 34건)이 넘어가 r_N이 **33.6%**25.4, 43.0로 커졌기 때문이다(사후 진단). 이를 완화하려던 클래스별 τ(G2)는 seed 42·43 병기에서 51.1·42.5배로 높았으나, 사전 등록한 새 **seed 44·45** 확인**(E13)**에서 **seed 44**는 **B0 76.0**배 → **G2 59.6**배**(−21.6%, F1 0.9908→0.9920, p = 0.34), seed 45**는 **42.4**배 → **54.4**배**(+28.4%, F1 0.9903→0.9913, p = 0.54)**로 두 seed 모두 세 조건을 채우지 못했다(기준 이상 1/2 seed, 유의성 0/2). G2는 B0가 나쁜 운영점을 고른 폴드만 메울 뿐 B0가 상한에 붙은 seed 44에서는 오히려 배수를 깎아, 넘김 판단 변형이 일관된 이득을 내지 못한다는 결과(5.2절)가 반복되었다. (단, 병기로 DL-A B0는 seed 44 +0.08%p·seed 45 −0.24%p를 기록해 네 seed 중 세 seed에서 DistilBERT 단독과 0.30%p 이내였으나, 판정 seed 43 실패는 그대로 보고한다.)
- **E14(**결합 학습 **k3** 출구 → **DistilBERT)**: 1차를 결합 학습한 k3 출구(0.159ms)로 바꾸면 같은 2차(DistilBERT)에서 합산 배수가 **29.3**배**(×2.40, seed 42)·23.8**배**(×2.27, seed 43)**로 커지고 정확도는 DL-A와 0.30%p 이내였다(두 seed 사전 기준 충족). 단, 이득의 상당 부분은 1차가 4배 싸진 데(0.159 대 0.604ms)서 나오며, DistilBERT 단독 대비로는 −0.31·−0.28%p이므로 DL-A 대비 판정이다.

5.4 왜 DistilBERT인가: CmdI 오답 분해(사후 분석)와 128 토큰 재구현(E12)

226,048행 합산 예측의 오류 분해(사후 분석)와 원 논문(WAMM[7]) 128 토큰 설정 재구현(E12) 결과는 Table 4와 같다.

Table 4. Pooled 5-fold error decomposition (226,048 rows) comparing char-CNN, DistilBERT (512 tokens, post-hoc), and the original 128-token setup (E12)

Metric / error count	s42 char-CNN	s42 DistilBERT(512)	s42 tok128 (E12)	s43 char-CNN	s43 DistilBERT(512)	s43 tok128 (E12)
Overall Macro-F1	0.9884	0.9914	0.9718 (−1.96 pp)	0.9862	0.9947	0.9777 (−1.70 pp)
Missed attack (Attack → Normal)	791	736 (−7.0%)	—	839	777 (−7.4%)	—
Real-user / scanner FPR	2.00% / 0.35%	1.82% / 0.43%	—	1.27% / 0.24%	1.09% / 0.21%	—
Attack-type confusion	**266**	**167 (−37.2%)**	—	**359**	**59 (−83.6%)**	—
└ SQLi → CmdI errors	**192**	**99 (−48.4%)**	**847**	**252**	**2 (−99.2%)**	**551**
CommandInjection F1	**0.9682**	**0.9784**	**0.9038**	**0.9618**	**0.9903**	**0.9284**

- **SQLi → CmdI** 오답 제거: DistilBERT(512)의 이득은 공격 차단(차단 실패는 두 seed 모두 약 7%만 감소)이 아니라 공격 종류 구분, 특히 **SQLi**를 **CmdI**로 오분류하던 오답의 제거**(seed 42 192→99**건**, seed 43 252→2**건**)**에서 나왔다. 가설로, char-CNN은 ;·|·' 같은 짧은 구분자에 반응하는 반면 DistilBERT는 512 토큰 어텐션으로 구분자 뒤가 SQL 절인지 OS 명령인지 본다. 교정된 오답에서 char-CNN 확신도 중앙값은 **0.775·0.766**으로 낮아(확신도 ≥ 0.99 비율은 전체 97.0% 대비 15.1%·10.7%), 캐스케이드**(B0)**는 시험 요청의 **0.7~0.8%**만 넘기면서 **SQLi→CmdI** 감소분의 절반 이상**(seed 42 192→123**건으로 **74%, seed 43 252→113**건으로 **56%)**을 물려받았다(사후 분석; 캐스케이드 차단 실패도 697·743건으로 단독 736·777건보다 적음).
- 원 논문 **128** 토큰 설정 재구현**(E12)**: 128 토큰으로 재구현한 DistilBERT는 512 토큰보다 유의하게 낮았고(seed 42 **−1.96%p**, seed 43 **−1.70%p**, 둘 다 p = 0.006, E12-1 채택), char-CNN 단독보다도 낮았다. 손실은 128 토큰에서 잘린 15,553행(정답률 0.994→0.957, 0.996→0.956; 안 잘린 행은 0.1%p 이내)에 몰려 CmdI F1이 0.9038·0.9284로 급락하고 SQLi→CmdI가 847·551건으로 폭증했다(E12-2). 배치 1에서는 요청의 93%가 128 토큰 이하라 지연도 512 토큰과 거의 같았다(46.10 대 46.08ms). char-CNN → DistilBERT-128은 DistilBERT-128 단독 대비 합산 **67.5**배**(+1.66%p)·76.3**배**(+0.84%p)**로 두 seed 모두 위였으나(E12-3), 이는 운영점이 사실상 넘김 0%여서 **"char-CNN** 단독이 **DistilBERT-128**보다 정확하고 **76**배 빠르다**"**를 다시 쓴 것이고, 512 토큰 판본은 seed 43에서 재현되지 않았다(E11).

견고성 점검 요약**(T5 · T10 · T11).** ① 시험 폴드 요청의 66.5%(CmdI 14.7%)가 학습 폴드에 같은 템플릿을 두나, 템플릿 묶음 분할(T5)에서 RGB CNN −0.06%p, char-CNN +0.10%p로 부풀림 기준(−2%p)에 해당하지 않았다. ② 4개 seed(42~45, T10)에서 RGB CNN은 0.8949~0.9229(표준편차 1.15%p, seed 42 최저), char-CNN은 0.9862~0.9901(0.18%p)이었다. ③ 봉인 시험 집합은 총 9회 참조됐고 설계를 바꾼 1차 학습 방식(클래스 가중치) 1건은 CV로 재확인(검증 분포 +27%, 실사용자 +5%, T11)하였다.

VI. 논의 및 한계

작동 조건과 실무 지침. 웹 공격 캐스케이드가 성립하려면 전처리를 포함한 $t_1 \ll t_2$와 낮은 실사용자 r_N이 동시에 충족되어야 한다. 이진 차단 위주이거나 0.3~0.8%p의 Macro-F1 차이가 허용된다면 **char-CNN** 단독 또는 결합 학습된 **DL-C(k3** 출구**)**가 DistilBERT보다 80~280배 빠르면서 차단 실패율도 대등하다. 반면 관제 자동화를 위해 SQLi와 CmdI 구분 정밀도가 필수적이면 512 토큰 DistilBERT가 유리하며, char-CNN → DistilBERT(DL-A, 합산 10.6~12.3배) 또는 k3 출구 → DistilBERT(E14, 합산 23.8~29.3배)가 지연을 한 자릿수 ms로 줄이는 절충안이 된다.

한계. 본 연구는 일곱 가지 한계를 갖는다. ① 지연은 데스크톱 CPU 1대·단일 스레드·배치 1·PyTorch 그대로만 쟀으며, 다중 스레드(B3 미실시)나 GPU 배치에서는 t_R/t_1 비율이 줄어 배수가 작아질 수 있다. ② 실사용자 정상이 폴 550건(폴드당 약 110건)뿐이라 실사용자 r_N의 폴드 간 분산이 크다. ③ DistilBERT 재구현은 원 논문의 128 토큰 대신 512 토큰·Adam을 주 기준으로 썼다(128 토큰은 E12에서 더 낮음을 확인). ④ 1차 확신 오답(≥ 0.99) 110건 감사에서 70.9%(대조 31.8%)가 라벨 불일치로 분류되었으나, 이는 대형 언어모델 1인 판정 기준 라벨 불일치 가능성이 높았다는 수준이며 2인 독립 감사(T8)는 생략하였다. ⑤ 고정 변형 기반 비용 공격 실험(E15)에서 URL 인코딩 1회(url_encode)만으로 DL-A(B0)의 공격 넘김 r_A가 0.27%→8.76%(약 **32**배), 공격 요청당 기대 지연이 0.73→4.64ms(**6.4**배) 늘어 DeepSloth[4] slowdown 공격이 모델 접근 없는 페이로드 변형만으로 이전됨을 확인하였다(2차가 약 76배 무거워 비용 증폭; 4개 변형 평균 r_A 배율은 B0 ×11.0, G2 ×2.78로 G2/B0 = 0.25여서 대가 조항은 미발동이나 E13에서 G2 이득이 미재현되어 안전 수단으로 확장 불가). 변형 공격도 4.64ms로 DistilBERT 단독(46.1ms)보다는 약 10배 싸서 $\pi=3.3\%$ 평균 비용 증가는 약 0.13ms이나, url_encode 시 **1**차 공격 판정률이 **99.4% → 86.7%**로 하락해 1차가 정상으로 확신한 변형 공격은 2차로 가지 않고 놓치는 탐지 회피 한계가 드러났다(적응형 공격 및 2차 포함 변형 정확도는 미측정). ⑥ 단일 데이터셋(SR-BH 2020, 정상 대부분이 스캐너 탐침) 평가이다. ⑦ 1-D CNN·선형 1차는 seed 42 1회 교차검증이다.

VII. 결론

본 논문은 바이트 이미지 CNN이 char-CNN보다 부정확하다는 음성 결과(RQ1)에서 출발해, 웹 공격 탐지 캐스케이드의 속도–정확도 교환과 넘김 판단의 한계(RQ5)를 SR-BH 2020에서 재평가하였다. 전처리 포함·무패딩 공정 지연에서 이미지 1차 캐스케이드는 char-CNN 단독의 **0.27**배로 역전되어 속도 이득을 철회했고, 확신도 게이트는 오라클보다 평균 **10.5**배 많이 넘겼으며 모든 게이트 변형(새 seed 44·45의 G2 포함)이 기각되었다. 반면 결합 학습된 char-CNN 내부 조기 종료(DL-C)는 모델 압축 효과로 **3.5~3.6**배 가속을 달성했고, char-CNN → DistilBERT(DL-A)는 SQLi→CmdI 오탐 감소분의 56~74%를 물려받으며 합산 **10.6~12.3**배 가속(k3 출구 결합 시 23.8~29.3배)을 얻었으나, 사전 정확도 기준은 seed 42(**−0.03%p**)에서만 충족하고 seed 43(**−0.32%p**)에서는 재현되지 않았다. 결국 캐스케이드의 성패는 넘김 점수가 아니라 종단간 지연 비율과 1차 판별력에 달려 있으며, 종단간 공정 지연·배포 기저율·실사용자 r_N·다중 seed 분산을 함께 보고할 때에만 유효한 평가가 성립한다.

References

[1] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, and K. Rieck, "Dos and don'ts of machine learning in computer security," Proceedings of the 31st USENIX Security Symposium, pp. 3971-3988, Aug. 2022.

[2] L. Nataraj, S. Karthikeyan, G. Jacob, and B.S. Manjunath, "Malware images: visualization and automatic classification," Proceedings of the 8th International Symposium on Visualization for Cyber Security (VizSec), pp. 1-7, Jul. 2011.

[3] W. Jitkrittum, N. Gupta, A.K. Menon, H. Narasimhan, A.S. Rawat, and S. Kumar, "When does confidence-based cascade deferral suffice?," Advances in Neural Information Processing Systems (NeurIPS), Dec. 2023.

[4] S. Hong, Y. Kaya, I.-V. Modoranu, and T. Dumitraş, "A panda? No, it's a sloth: slowdown attacks on adaptive multi-exit neural network inference," International Conference on Learning Representations (ICLR), May 2021.

[5] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter," arXiv:1910.01108, Oct. 2019.

[6] F. Sureda Riera, J. Bermejo Higuera, J.R. Bermejo Higuera, J.J. Martínez Herraiz, and J.A. Sicilia Montalvo, "A new multi-label dataset for web attacks CAPEC classification using machine learning techniques," Computers & Security, 120, 102788, Sep. 2022.

[7] H. Osama, O. Elebiary, Y. Qassim, M. Amgad, A. Maghawry, A. Saafan, and H. Ghalwash, "Enhanced web payload classification using WAMM: an AI-based framework for dataset refinement and model evaluation," arXiv:2512.23610, Dec. 2025.

[8] Y. Wu, D. Zou, S. Dou, W. Yang, D. Xu, and H. Jin, "VulCNN: an image-inspired scalable vulnerability detection system," Proceedings of the 44th International Conference on Software Engineering (ICSE), pp. 2365-2376, May 2022.

[9] E. Eroğlu Demirkan and M. Aydos, "Enhancing malware detection via RGB assembly visualization and hybrid deep learning models," Applied Sciences, 15(13), 7163, Jun. 2025.

[10] J.R. Tadhani, V. Vekariya, V. Sorathiya, S. Alshathri, and W. El-Shafai, "Securing web applications against XSS and SQLi attacks using a novel deep learning approach," Scientific Reports, 14, 1803, Jan. 2024.

[11] X. Zhang, J. Zhao, and Y. LeCun, "Character-level convolutional networks for text classification," Advances in Neural Information Processing Systems (NIPS), pp. 649-657, Dec. 2015.

[12] K. Luo, Y. Xie, S. Zhao, and J. Pan, "WADBERT: dual-channel web attack detection based on BERT models," arXiv:2601.21893, Jan. 2026.

[13] M. Bitussi and R. Doriguzzi-Corin, "X-WAD: explainable web anomaly detection," arXiv:2608.27172, Aug. 2026.

[14] X. Wang, D. Kondratyuk, E. Christiansen, K.M. Kitani, Y. Movshovitz-Attias, and E. Eban, "Wisdom of committees: an overlooked approach to faster and more accurate models," International Conference on Learning Representations (ICLR), Apr. 2022.

[15] D. Hendrycks and K. Gimpel, "A baseline for detecting misclassified and out-of-distribution examples in neural networks," International Conference on Learning Representations (ICLR), Apr. 2017.

[16] K. Tasdemir, R. Khan, F. Siddiqui, S. Sezer, F. Kurugollu, S.B. Yengec-Tasdemir, and A. Bolat, "Advancing SQL injection detection for high-speed data centers: a novel approach using cascaded NLP," arXiv:2312.13041, Dec. 2023.

[17] Y. Geifman and R. El-Yaniv, "Selective classification for deep neural networks," Advances in Neural Information Processing Systems (NIPS), pp. 4878-4887, Dec. 2017.

[18] H. Mozannar and D. Sontag, "Consistent estimators for learning to defer to an expert," Proceedings of the 37th International Conference on Machine Learning (ICML), pp. 7076-7087, Jul. 2020.

[19] S. Enomoto and T. Eda, "Learning to cascade: confidence calibration for improving the accuracy and computational cost of cascade inference systems," Proceedings of the

AAAI Conference on Artificial Intelligence, 35(8), pp. 7331-7339, Feb. 2021.

[20] C. Guo, G. Pleiss, Y. Sun, and K.Q. Weinberger, "On calibration of modern neural networks," Proceedings of the 34th International Conference on Machine Learning (ICML), pp. 1321-1330, Aug. 2017.

[21] S. Teerapittayanon, B. McDanel, and H.T. Kung, "BranchyNet: fast inference via early exiting from deep neural networks," Proceedings of the 23rd International Conference on Pattern Recognition (ICPR), pp. 2464-2469, Dec. 2016.

[22] K. Lee, K. Lee, H. Lee, and J. Shin, "A simple unified framework for detecting out-of-distribution samples and adversarial attacks," Advances in Neural Information Processing Systems (NeurIPS), Dec. 2018.

[23] P.F. Jaeger, C.T. Lüth, L. Klein, and T.J. Bungert, "A call to reflect on evaluation practices for failure detection in image classification," International Conference on Learning Representations (ICLR), May 2023.

[24] V.J. Reddi, C. Cheng, D. Kanter, P. Mattson, G. Schmuelling, C.-J. Wu, B. Anderson, M. Breughe, M. Charlebois, W. Chou, R. Chukka, C. Coleman, S. Davis, P. Deng, G. Diamos, J. Duke, D. Fick, J.S. Gardner, I. Hubara, S. Idgunji, T.B. Jablin, J. Jiao, T.S. John, P. Kanwar, D. Lee, J. Liao, A. Lokhmotov, F. Massa, P. Meng, P. Micikevicius, C. Osborne, G. Pekhimenko, A.T.R. Rajan, D. Sequeira, A. Sirasao, F. Sun, H. Tang, M. Thomson, F. Wei, E. Wu, L. Xu, K. Yamada, B. Yu, G. Yuan, A. Zhong, P. Zhang, and Y. Zhou, "MLPerf inference benchmark," Proceedings of the 47th Annual International Symposium on Computer Architecture (ISCA), pp. 446-459, May 2020.

[25] S. Axelsson, "The base-rate fallacy and its implications for the difficulty of intrusion detection," Proceedings of the 6th ACM Conference on Computer and Communications Security (CCS), pp. 1-7, Nov. 1999.

[26] C. Nadeau and Y. Bengio, "Inference for the generalization error," Machine Learning, 52(3), pp. 239-281, Sep. 2003.

[27] L.I. Kuncheva and C.J. Whitaker, "Measures of diversity in classifier ensembles and their relationship with the ensemble accuracy," Machine Learning, 51(2), pp. 181-207, May 2003.